# CALC WARS: A Card Game with a Hierarchical Neuro-Symbolic Framework of AI opponents

Westlake University

**Abstract**

We introduce CALC WARS, a competitive two-player card game derived from the traditional "24 Points" logic. We developed a human-expert level AI using a hierarchical architecture that combines a symbolic solver with a neural "Commander Net". The model was trained using a hybrid approach of Proximal Policy Optimization and Behavior Cloning. Our AI achieves SOTA performance, maintaining an average win rate of >70% against scripted strategies and >80% against human testers. We open-source the game engine and training framework to facilitate further research in neuro-symbolic reinforcement learning. `https://github.com/2001Haru/CALC-WARS`

## 1 Introduction

CALC WARS is a competitive two-player card game derived from the traditional '24 Points' logic. It is designed around the principle that a local gameplay optimum does not necessarily yield a global strategic advantage. To achieve this, the game incorporates a system of specialized skill cards that allow players to manipulate hand cards and health points, significantly increasing the strategic depth beyond simple arithmetic calculation.

To provide a human-expert level opponent for CALC WARS, we developed an agent utilizing a hierarchical neuro-symbolic architecture. Training a standard reinforcement learning agent for this environment is challenging due to the vast combinatorial space of arithmetic expressions and the complexity of long-term resource management. Our approach addresses these hurdles by decoupling symbolic calculation from strategic planning. By utilizing a dedicated symbolic solver to handle reachable expressions, we allow a lightweight 4.5M parameter 'Commander Net' to focus exclusively on high-level decision-making.

We evaluate our agent against both scripted heuristic strategies and human testers. The model achieves state-of-the-art performance, maintaining win rates exceeding 80% against human players. Furthermore, we explore the model's generalization boundaries through a Red Teaming mechanism using adversarial scripts. This analysis identifies specific out-of-distribution (OOD) vulnerabilities, hence significantly improved the model's game robustness through providing complemental training samples.

## 2 Main Result

### 2.1 Core Gameplay

Each player starts with 120 HP. Players need to use the cards in their hands to form valid mathematical expressions to perform actions. Specific expression results can trigger special effects. Forming Square, Cube, Factorial, 24, 0, or 1 results in different skill cards:

- **Square**: DRAW card – Draws two random numbers and one symbol.
- **Cube**: STEAL card – Randomly steals two cards (one if hand difference $> 6$).

- **Factorial**: HEAL card – Restores 20 HP and draws one card.
- **24**: Two SHIELD cards – Blocks any single attack.
- **0**: RUIN card – Randomly destroys three cards in the opponent's hand.
- **1**: PIERCE card – Destroys all opponent shields.

Moreover, we design special targets with seven numbers, where players calculate the numbers in different areas to inflict varying levels of damage on their opponent. To enhance strategic depth, the game features a unique turn and round system. We adopted the traditional "Initiative by Passing mechanism", meaning that the player who ends the current round first gains the initiative for the next round. The victory condition of the game is to deplete the opponent's health bar

Regarding the interactive experience of the game, we have designed rich sound effects and fully replicated the art style of the Windows 7 operating system, providing dynamic auditory and visual feedback.

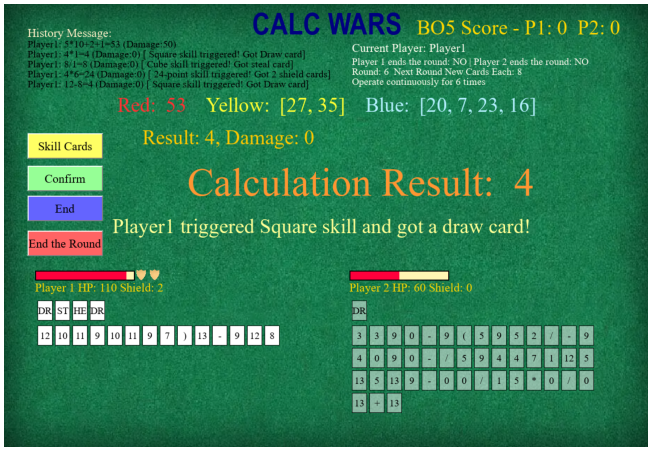



Figure 1: UI and Gameplay Interface
*Visual representation of the game environment.*

## 2.2 AI Opponent

The game's AI Battle mode comes with AIs of varying difficulty levels, allowing players to freely choose according to their preference. The highest difficulty AI model reaches human-expert performance levels. We wrote six generations of scripts to train and test the performance. The model achieved high win rates against scripts opponents and human testers.

Table 1: AI Performance Statistics against Various Opponents

| **Opponent** | **Wins** | **Losses** | **Win Rate** |
|---|---|---|---|
| Demon_v1 (Balanced/Resource Control) | 2645 | 355 | 88.17% |
| Demon_v2 (Aggressive/Offense) | 2746 | 254 | 91.53% |
| Demon_v3 (Conservative/Defense) | 2759 | 241 | 91.97% |
| Demon_v4 (Novel Strategy/Generalization) | 2827 | 173 | 94.23% |
| Demon_v5 (Red Teaming/Adversarial) | 2166 | 834 | 72.20% |
| Demon_v6 (Red Teaming/Adversarial) | 1944 | 1056 | 64.80% |
| Human Testers | 20 | 4 | 83.33% |
| BO5 with Human Testers | 8 | 2 | 80.00% |
| BO7 with Human Testers | 9 | 1 | 90.00% |

# 3 Methodology

## 3.1 Game Environment and Design Philosophy

We implemented CALC WARS using the Pygame framework to provide a native Python environment compatible with PyTorch for AI training. Two core principles guided the game design: "Local optimum is not Global optimum" to ensure strategic depth, and "Simple but infinite fun" to lower entry barriers. To maintain competitive balance, we implemented iterative rule refinements, such as a protection mechanism for the STEAL skill card to prevent a dominant "Steal Meta". The UI/UX follows a Windows 7-inspired aesthetic, utilizing Ocenaudio for sound asset editing to provide dynamic auditory feedback.

## 3.2 AI Opponent

The construction of the AI is the most critical engineering component of this project. The final version of the model underwent over 40 hours with three stages: Pre-training, Main Training and Post-training. Our training architecture draws heavily from the experience of the DeepMind team in training `AlphaStar`, employing an industrial-grade training paradigm.

### 3.2.1 Training Feasibility Analysis

The training environment for CALC WARS presents unique structural advantages and high-level strategic hurdles:

- **Environmental Advantages:** The game is modeled as a fully Markovian, perfectinformation environment with a well-defined discrete action space, simplifying state representation and transition modeling.
- **Strategic Hurdles:** The "Local is not Global optimum" principle necessitates complex credit assignment and reward function design, as immediate arithmetic gains do not always translate to long-term victory.
- **Arithmetic Bottleneck:** Learning symbolic logic directly through neural weights is resource-intensive due to the vast combinatorial space of possible expressions.

### 3.2.2 Model Architecture

To address the challenges of arithmetic complexity and long-term strategic planning, we implement a hierarchical neuro-symbolic architecture. This design decouples low-level symbolic execution from high-level strategic reasoning.

**Symbolic Solver (The Executor)** The low-level Solver functions as a deterministic execution engine. Utilizing NumPy-vectorized operations, it exhaustively scans the current hand state to identify all reachable arithmetic targets and their corresponding target expressions. By offloading arithmetic verification to this symbolic component, the neural network is liberated from learning basic mathematical identities, allowing it to focus exclusively on the game's competitive meta-strategy.

**Commander Net (The Strategist)** The strategic core, or "Commander Net," is an 8-layer Residual MLP with approximately 4.5M parameters. The architecture is detailed as follows:

**State Vector and Action Space** The model receives a 92-dimensional state vector, encoding comprehensive game information including health points (HP), shield counts, hand card distributions, and skill card inventories for both players.

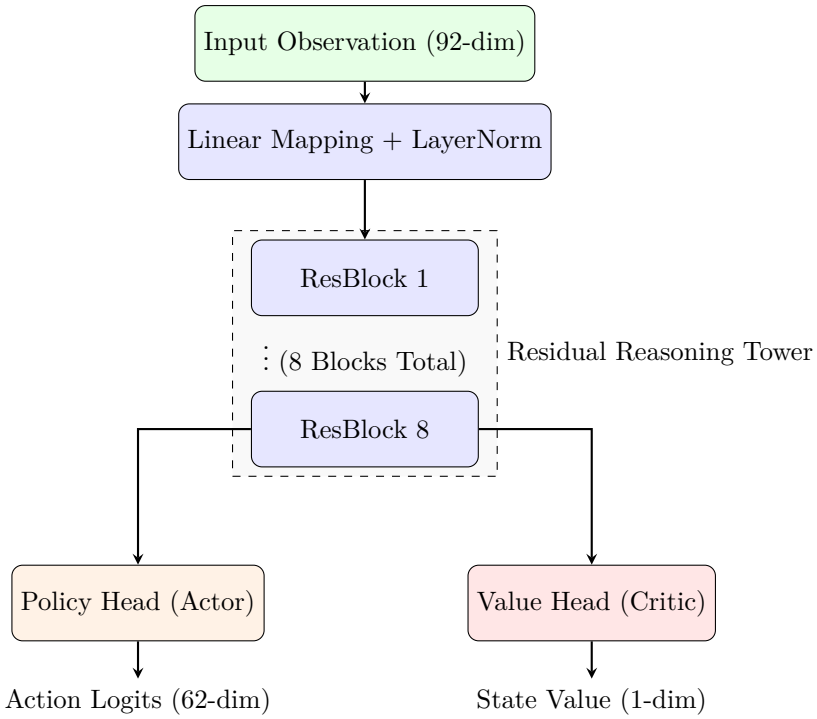



Figure 2: Hierarchical Architecture of the Commander Net

To bridge the gap between the neural commander and the symbolic executor, we employ **Oracle Action Masking**. The Solver generates a real-time boolean mask based on the current hand cards; this mask is applied to the policy logits, effectively zeroing out the probability of any unreachable arithmetic targets or illegal actions, thereby forcing the model to explore only the valid strategic state space.

## 3.3 Training Stages

The project primarily adopts a 'League Training' mode, with opponents drawn from our designed script experts and historical models in the opponent pool. The training algorithm employs a hybrid approach combining Proximal Policy Optimization and Behavioral Cloning.

**Decision Tree Expert Opponents** We developed six generations of **Demon** scripts. **V1** replicates the dominant playstyle identified by our team. **V2** is more offensive, while **V3** is more defensive. **V4**, **V5** and V6 are exploitation scripts designed in post-training to target model weaknesses for adversarial training. **Demon_V1** served as the primary guiding model, acting as the cloning strategy during the pre-training stage. The other models were introduced to enhance the model's strategic adaptability.

**Opponent Pool Mechanism** We maintained an opponent pool where sampling is weighted based on win rates to prioritize strong opponents. We utilized a `FIFO` elimination method with a minimum "age" threshold to prevent the rapid burial of rare strategy models. This pool prevents Nash equilibrium stagnation and contributes significantly to opponent diversity.

### 3.3.1 Pre-training via Behavior Cloning (BC)

In the pre-training stage, to address the cold-start problem in PPO training, the model directly cloned the strategy of **Demon_V1**. This phase took 8 hours and 100k episodes. By the end, the model achieved a draw-level win rate against **V1** and over 60% against **V2** and **V3**.

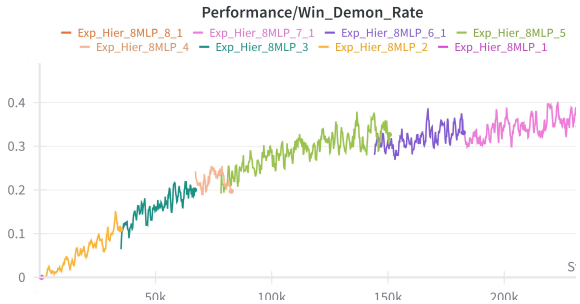
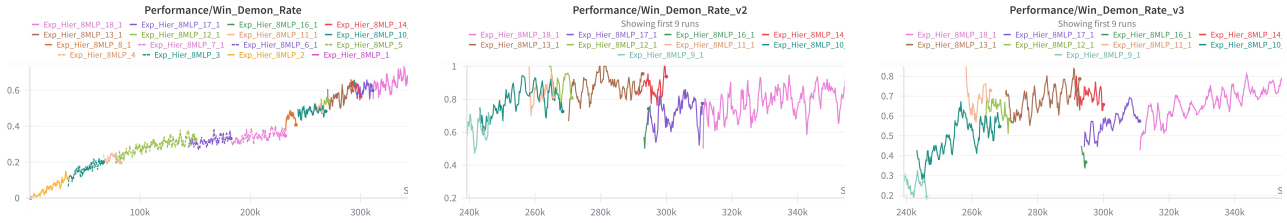



Figure 3: Model Wins DemonV1 Rate

### 3.3.2 Main Training: Hybrid PPO and BC

From an engineering perspective, we combined **PPO** and **BC**. Fresh trajectories were assigned to on-policy PPO for loss calculation, while old but valuable data from Prioritized Experience Replay (`PER`) were used for off-policy BC loss. The total loss is a weighted sum controlled by a coefficient. Trajectories in which the model defeats powerful opponents are stored in a PER buffer, significantly improving the efficiency of the exploration for optimal strategies. In main training, the BC weight was gradually decreased to allow free exploration of superior strategies.

$$\mathcal{L}_{total} = \mathcal{L}_{PPO} + \alpha \cdot \mathcal{L}_{BC}(\mathcal{D}_{PER}) \tag{1}$$




(a) Win Rate to DemonV1 (b) Win Rate to DemonV2 (c) Win Rate to DemonV3

Figure 4: Comparison of different Demon Version Opponents

### 3.3.3 Post-training: Exploitation and Adversarial Training

Borrowing from the `FAR.AI` team's approach of training adversarial model for Go AI `KataGo`, we designed **Demon_V4 V5** and **V6** for late-stage robustness tuning, which successfully identified strategic weaknesses and decision hallucinations. These expert opponents employ seemingly irrational and simple strategies that rarely appear in normal play, which might even lose to human novices. However, the model's initial win rate against these opponents are critically low, quite near 50%. In post-training stage, the model's opponents will only come from the opponent pool and **Demon_V4 V5** and **V6**. The goal is to address the model's original overfitting issues with previous expert strategies, and to optimize its strategy for handling less common branches of gameplay. Through this adversarial training, the model's game robustness has significantly improved, and the problem of strategy collapse in deep learning has been greatly alleviated.

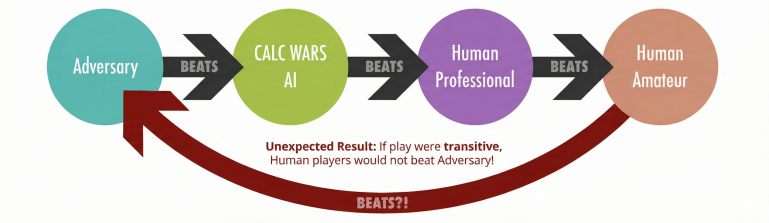
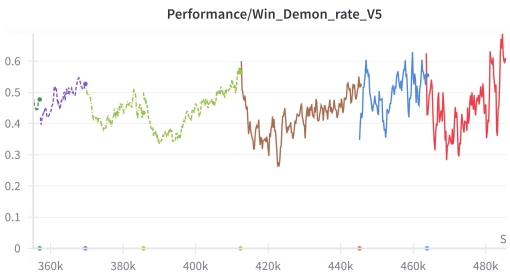
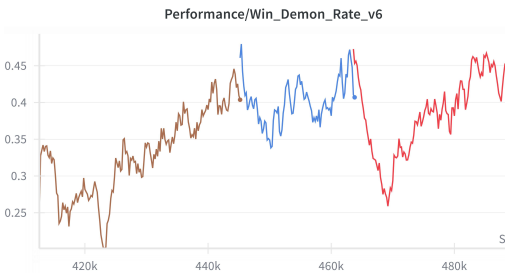



Figure 5: Food Chain in Gameplay




(d) Win Rate to DemonV5




(e) Win Rate to DemonV6

Figure 6: Comparison of different Demon Version Opponents

# 4 Discussion

## 4.1 Pure Neural Networks and Hierarchical Mixture of Experts

In the early stages of the project, we experimented with a pure neural network architecture, but the results fell far short of expectations. A pure neural network architecture required the model to compose an expression through multiple operations, rather than performing it in a single step as in a hierarchical architecture. This design of multi-step operation action space led to a catastrophic credit assignment problem. Another approach to designing the action space involved enumerating all possible expression combinations for the model to choose from, but this resulted in an explosion of the action space size and non-fixed dimensionality issues. Ultimately, with the multi-step operation action space design, the model only reached the level of a novice human player, exhibiting shallow strategic awareness and even failing to guarantee that all expressions were valid. All of these factors prompted us to shift toward the current hierarchical architecture.

# 5 Limitations and Future Work

Future research will focus on automated adversarial play generation. We also intend to scale model parameters and training volume. Here're the major limitations of the project:

- **Nascent Meta of the Game:** Unlike Go, CALC WARS has a small player base and fewer established manuals. Even as developers, we discovered emergent AI strategies—such as synthesizing Square/DRAW cards to manipulate hand differences, evading the STEAL protection mechanism—that we had not anticipated. Our understanding of the game's full potential is limited, which hence constrains the training of our model.
- **Efficiency-Performance Trade-off Analysis:** The model's size fundamentally limits the upper bound of its decision-making capability. In practice, training a 4.5M-parameter model to explore and find optimal solutions in a strategic game's decision space is extremely challenging and demands highly precise and refined training schemes. We are eager to scale up the model's parameter count to achieve stronger strategic

capabilities, despite our severe limitations in training compute resources. Nevertheless, as a small-scale model, its training performance has already met and even surpassed the project's initial expectations.

- **The fragility of static policies against adaptive human intelligence:** As a static strategy, the model is highly susceptible to being studied, and humans excel at dissecting such fixed strategies. After adapting to and analyzing the model's tactics, human experts can significantly increase their win rate against it to about 70%. However, our original intention in developing this AI for the game was not to defeat all human players, but to provide players with a powerful training partner and a formidable opponent.

# 6 Conclusion

CALC WARS demonstrates that hierarchical architectures can achieve expert-level performance in complex games with lightweight parameters. Our adversarial testing provided insights into the limitations of deep learning strategy collapse. While we focused on CALC WARS, we hypothesize these results will apply more generally and these methods can solve zero-sum two-player continuous environment which can be simulated in parallel across hundreds of thousands of instances.